

Assignment 4 — Task 1: CIFAR-10 Classification

Applied Deep Learning, Spring 2026 · Convolutional Neural Networks

Ron Kadosh · Naim Elijah

Setup

We follow the PyTorch CIFAR-10 tutorial. RGB 32×32 images are normalized per channel with mean 0.5 and std 0.5. The network is the tutorial CNN: $Conv(3 \rightarrow 6, 5 \times 5) \rightarrow ReLU \rightarrow MaxPool(2) \rightarrow Conv(6 \rightarrow 16, 5 \times 5) \rightarrow ReLU \rightarrow MaxPool(2) \rightarrow FC(400 \rightarrow 120) \rightarrow FC(120 \rightarrow 84) \rightarrow FC(84 \rightarrow 10)$. Trained with cross-entropy loss and SGD ($lr = 0.001$, momentum = 0.9) for 2 epochs, batch size 4, on an NVIDIA RTX 3060 Ti.

Accuracy

56.71%

TRAINING ACCURACY

55.24%

TEST ACCURACY

Both far exceed the 10% random-guess baseline for 10 classes. Per the assignment, exact reproduction of the tutorial's numbers is not required; training was limited to 2 epochs.

Sample test images with predicted labels

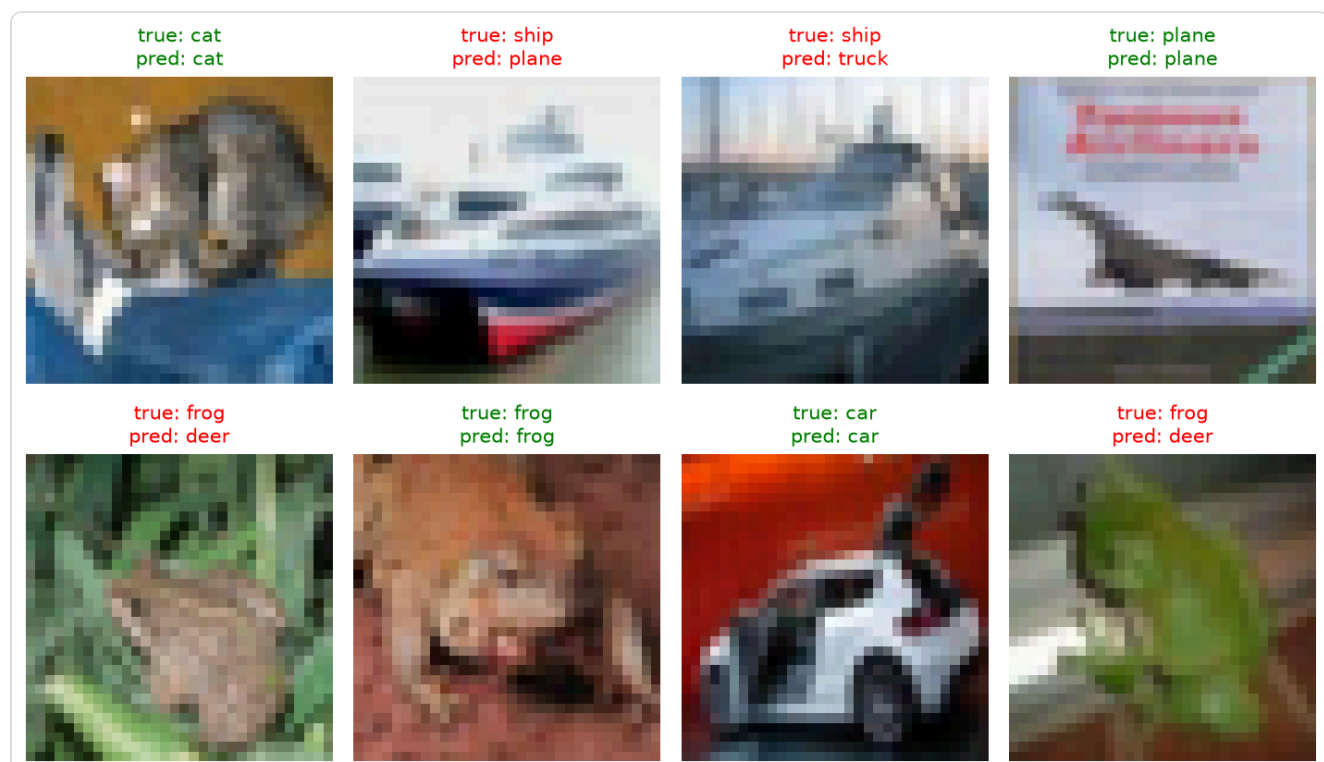


Figure 1. Eight test images, each labelled with its true class and the model's prediction. Titles are **green** when correct and **red** when wrong. Errors tend to occur between visually similar classes (e.g. ship→truck, frog→deer).

Per-class accuracy

Class	Accuracy
plane	69.6%
car	56.2%
bird	33.9%
cat	36.3%
deer	40.5%
dog	57.4%
frog	62.0%
horse	53.5%
ship	63.7%
truck	79.3%

Man-made classes such as truck, plane, and ship are recognized most reliably, while several animal classes (bird, cat, deer) are hardest — the usual CIFAR-10 pattern, since these natural categories share textures, poses, and backgrounds that are easily confused.

Assignment 4 — Task 2: Deconvolutional Model

Applied Deep Learning, Spring 2026 · Joint classification + reconstruction

Ron Kadosh · Naim Elijah

Architecture

We **extend the Task 1 CNN**: its encoder (two Conv→ReLU→MaxPool blocks) and classifier head (the three FC layers) are kept unchanged, and a **decoder head** is added that mirrors the encoder to reconstruct the input. Both heads branch from the same latent $z^{(2)}$ ($16 \times 5 \times 5$): the classifier maps it to the 10 logits, the decoder maps it back to an image $\tilde{x} \in \mathbb{R}^{3 \times 32 \times 32}$.

ENCODER (reused from Task 1)	DECODER (new, mirrors encoder)
3x32x32	3x32x32 = $\tilde{x}$ tanh(deconv1)
conv1(5) -> 6x28x28	6x28x28 unpool1 (uses pool-1 indices)
ReLU, pool -> 6x14x14 = $z(1)$	6x14x14 ReLU(deconv2)
conv2(5) -> 16x10x10	16x10x10 unpool2 (uses pool-2 indices)
ReLU, pool -> 16x5x5 = $z(2)$	16x5x5 = $z(2)$ (decode starts here)
+-- FC 400->120->84->10 = $\tilde{y}$ (logits, the Task-1 head)	

Each encoder block *Conv*→*ReLU*→*MaxPool* is inverted, in reverse order, as *MaxUnpool*→*ConvTranspose2d*→*ReLU*. **MaxUnpool2d** needs the pooling *indices* from the matching *MaxPool2d*(*return_indices=True*) to place each value back where it came from; **ConvTranspose2d** with the same 5×5 kernel undoes the conv's spatial shrink (28→32, 10→14). The final layer uses **tanh** (not ReLU) so the output matches the normalized input range [−1, 1].

Training setup & loss

Trained from scratch with the same regime as Task 1 — SGD (*lr* = 0.001, momentum = 0.9), batch size 4, cross-entropy — but for **10 epochs** so the accuracy curves have shape. The network optimizes the combined loss

$$L = L_{\text{CE}}(\tilde{y}, y) + \lambda \cdot L_{\text{rec}}(\tilde{x}, x), \quad L_{\text{rec}} = (1/3) \sum_{i=1..3} \|\tilde{x}_i - x_i\|_F^2$$

i.e. the per-channel squared error summed over pixels, averaged over the 3 channels (and over the batch).

Choice of $\lambda = 0.01$. The two terms sit on very different scales: cross-entropy is ≈ 1 , while the raw reconstruction term settles around 64 (see the right panel of Figure 1). Setting $\lambda = 0.01$ brings the weighted reconstruction term $\lambda \cdot L_{\text{rec}} \approx 0.64$ to the *same order of magnitude* as L_{CE} , so neither objective dominates: the decoder learns to reconstruct without the reconstruction gradient swamping classification. Larger λ (e.g. 0.05) made the reconstruction term dominate and depressed accuracy.

Results

69.34%

FINAL TRAINING ACCURACY

59.98%

FINAL TEST ACCURACY

0.01

Λ

Test accuracy reaches **60.0%** — on par with the classification-only Task 1 model, confirming that adding the reconstruction objective does not harm classification.

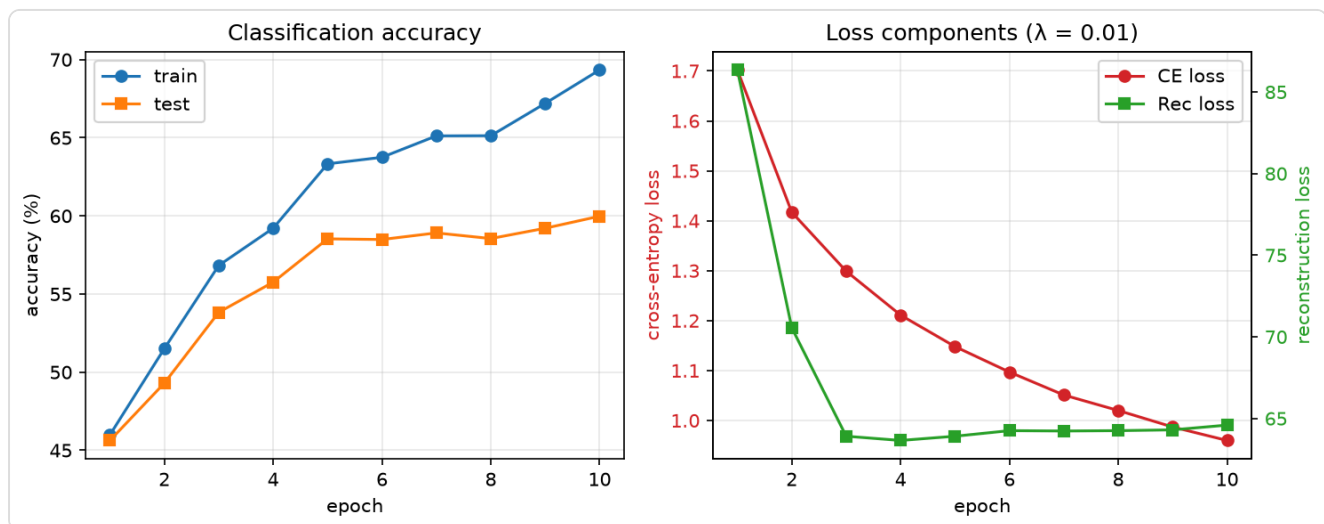


Figure 1. *Left:* train and test classification accuracy per epoch — both rise smoothly; the widening train/test gap is the usual mild overfitting of this small network. *Right:* the two loss components (twin axes, since they differ $\sim 60\times$ in scale). Reconstruction loss drops sharply in the first 3 epochs to a floor near 64 — the limit imposed by the lossy MaxUnpool path — while cross-entropy keeps declining.

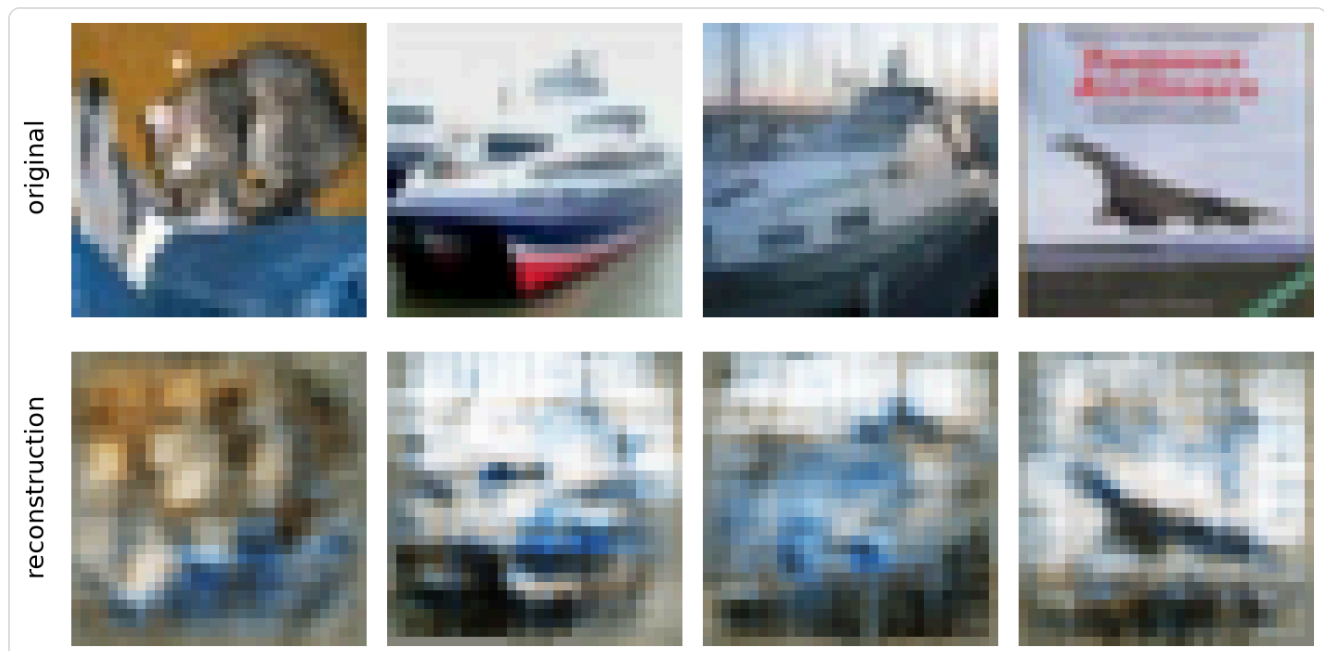


Figure 2. Four test images (top) and their reconstructions (bottom), un-normalized for display. The decoder recovers global colour and coarse object shape and position (cat on blue cloth, ship hulls, the Concorde silhouette) but loses fine detail — expected, because MaxUnpool restores only the pooled maxima and zeroes everything else, and the latent is just $16\times 5\times 5$.

Per-class test accuracy

Class	Accuracy
plane	65.3%
car	68.1%
bird	36.5%
cat	42.9%
deer	62.2%
dog	44.7%
frog	77.0%
horse	66.0%
ship	66.8%
truck	70.3%

The same pattern as Task 1 persists: man-made and distinctive classes (frog, truck, car, ship) are easiest, while visually cluttered animal classes (bird, cat, dog) remain hardest.

Assignment 4 — Task 3: Latent Representation Analysis

Applied Deep Learning, Spring 2026 · What each latent channel captures

Ron Kadosh · Naim Elijah

Method

Using the decoder trained in Task 2, we probe the learned representation of **one train image** (a horse) and **one test image** (a plane). For a chosen latent and channel c , we keep channel c and set every other channel to zero, then decode back to image space. Whatever survives in the reconstruction is what that single channel contributes.

- $\mathbf{z}^{(1)}$ — the conv-1 latent (6 channels, $6 \times 14 \times 14$). We show **all 6** channels, decoded through the conv1-block inverse (decode(level=1)).
- $\mathbf{z}^{(2)}$ — the conv-2 latent (16 channels, $16 \times 5 \times 5$). We show the **3 most active** channels (indices [8, 11, 12], the same for both images so columns are comparable), decoded through the full path.

$\mathbf{z}^{(1)}$: low-level colour / intensity features

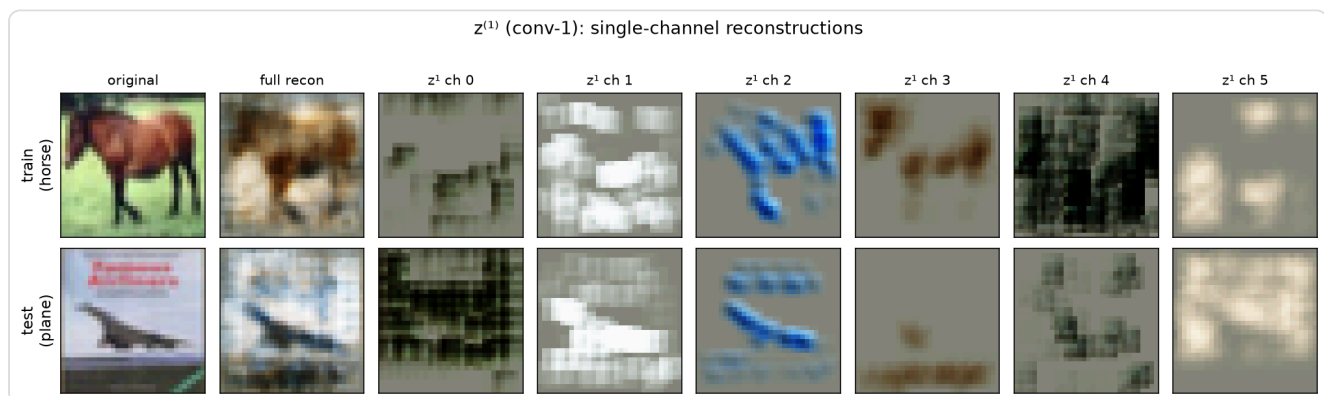


Figure 1. Each conv-1 channel kept in isolation (others zeroed), decoded to image space. Columns: original, full reconstruction, then channels 0–5.

Channel 1 marks bright, high-luminance highlights; channels 0/4 respond to dark, low-luminance background and shadow (grass); channels 2/5 trace the object body and outline. Only channel 3 shows a genuine *colour* correlation — strong on the brown horse, nearly absent on the plane. Overall the conv-1 channels act as low-level edge / luminance / colour-contrast detectors, each retaining the object's shape because the latent is still 14×14 .

$\mathbf{z}^{(2)}$: higher-level spatial structure

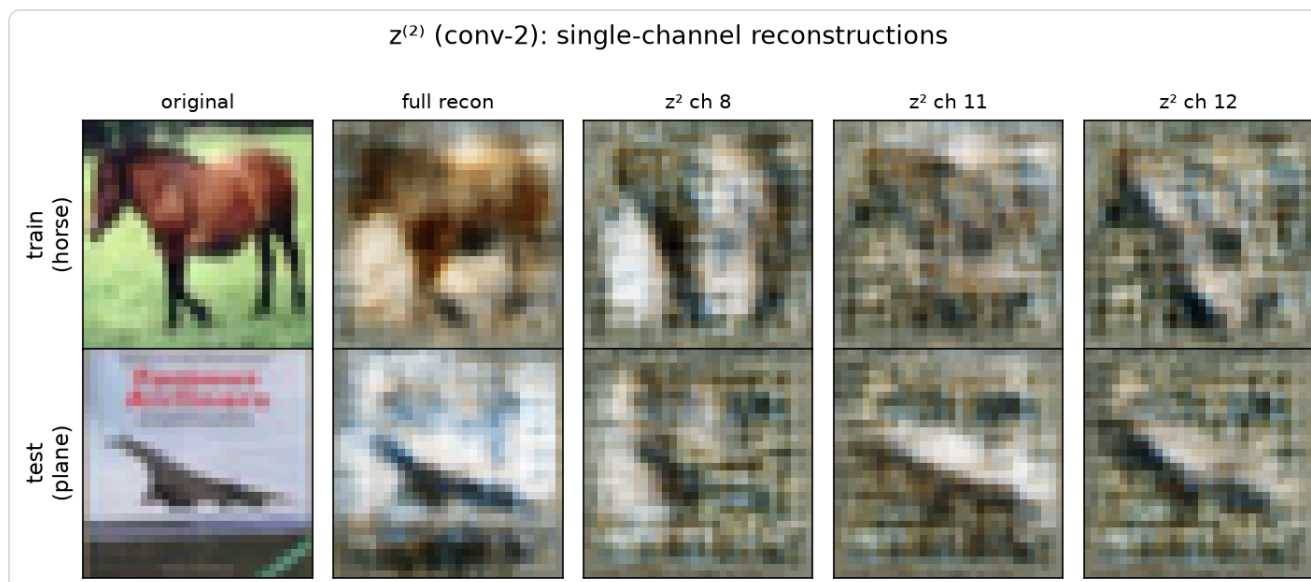


Figure 2. Three conv-2 channels kept in isolation, decoded through the full path. Columns: original, full reconstruction, then channels 8, 11, 12.

The conv-2 channels are **more abstract and far coarser**. The latent is only 5×5 , so a single channel can no longer reproduce a clean colour image; instead each channel encodes a **coarse spatial / figure-ground layout** rendered as blocky, mottled texture. **Channel 8** recovers the bright object-on-background structure (the horse's body, the plane's fuselage), while the others carry textured background and positional patterns. No single $z^{(2)}$ channel reconstructs the image on its own — the information is **distributed** across channels, which is why zeroing all but one leaves only a rough sketch.

Discussion

The two latents show the expected **feature hierarchy** of a CNN. $z^{(1)}$, close to the input, decomposes the image into low-level *colour and intensity* primitives that still retain spatial detail. $z^{(2)}$, deeper and heavily downsampled, trades colour fidelity and resolution for more *abstract, spatially-organised* structure (object vs. background, coarse part layout) that is distributed across many channels — exactly the representation a classifier head needs. The same channels play the same roles on both the train and test image, indicating the features generalise rather than memorise.